

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»

КОМП'ЮТЕРНИЙ ПРАКТИКУМ №1

з навчальної дисципліни *«Спеціальні розділи обчислювальної
математики»*

на тему: *«Багаторозрядна арифметика»*

Виконав: ст. НН ФТІ гр. ФБ-45

Гінкул Максим Вікторович

Багаторозрядна арифметика

Мета роботи

Отримання практичних навичок програмної реалізації багаторозрядної арифметики; ознайомлення з прийомами ефективної реалізації критичних по часу ділянок програмного коду та методами оцінки їх ефективності.

А) Згідно варіанту розробити клас чи бібліотеку функцій для роботи з m -бітними цілими числами. Бібліотека повинна підтримувати числа довжини до 2048 біт.

Для представлення 2048-бітних беззнакових цілих чисел у структурі `BigInt` використано фіксований стековий масив `uint32_t digits[64]`, оскільки $64 \times 32 = 2048$ біт. Число розбивається на блоки за схемою *Little-Endian*.

Повинні бути реалізовані такі операції:

1) переведення малих констант у формат великого числа (зокрема, 0 та 1);

Переведення 32-бітних скалярних констант у формат великого числа реалізовано через конструктор `BigInt(uint32_t val)`. Значення константи записується в нульовий (наймолодший) елемент масиву `digits[0] = val`, а всі інші 63 старші машинні слова занулюються в циклі `digits[i] = 0`. Для зручності визначено статичні методи `BigInt::zero()` та `BigInt::one()`, які повертають об'єкти, сконструйовані від скалярів 0 та 1 відповідно.

2) додавання чисел;

Додавання реалізовано у методі `add` класичним стовпчиковим алгоритмом із послідовним проходом по всіх 64 машинних словах від молодших розрядів до старших. Для запобігання апаратному переповненню проміжна сума обчислюється у 64-бітному типі. Молодші 32 біти результату записуються у відповідне слово результуючого числа, а біт переповнення для наступної ітерації виділяється зсувом `carry = temp >> 32`. Метод повертає фінальне значення `carry` (0 або 1), яке сигналізує про вихід суми за межі 2048-бітної сітки, а оператор `+` виконує цю операцію за модулем 2^{2048} .

Приклад:

```
a = eec2cfb3ea7ab2bb0643950e5e829d6125a6eba72e6b0c8487aa7e1f44e4d7e7c2a9e589949c1d10aa0ab78956f97debdeb3d82d983199
3c93fdef251f6dc1c92fd9a5955603371185f73e2b7faeacc730428e4119f88ca6414198555eaaaf24284d495b6b5c61481791b81ad15c537ac3
51a93bb3cbd7ef9a0f6fc4179c41cf2a1513b3f1ceff196dac32035518ff51bab79dc8c8493285353814549bbbaea147de43740a4cef3ee2f7c4d
96286ebaf015d1f4bfcd6d16ab5087be355e58afa40fe7b69f3fb371477eb5515219a8c6fd22089f98db08e423ef8d2d708d11adeeb15a1515e
404360ee2b2bef60715bc674d7a04b4ad3ead116d61c56f68a07
b = f1052a6a01d365ca23eb1674bb4c4ae15a0e13c66572d0d13533cf3c0d8caafab1ca1d6b9d772ee47f5aae94c68b544dc3526e6b737072fe
ebf8fd47083aa0ce6074f1fb8646d9a39d7dce52a570f5093ba9467aebd2c0f6e6a243f4b203c0e661fb42f395b7c386d678b03d3896ea028912
da5259c5cccfad727c2abe7d54dab3636ce0554e5372510428a56353388faeb32212301a73063c3ed042e5f8c81296dbaff5c88b0eb42de74cb75
329cd7cd96a36fd5d7a88901529e981a7de2da3ddc55770ab0c3b7dcb0c22b7202e03e1441c2771f24a7520b4f602a0e4f6326bd4b3a43a7917a
075d83fcec5dec7b1d4c0f29a6ae556dc6e5edc47d821853531d
a + b (hex) = dfd3ed39b5bde07cdef15a09c9aacd7ebb33bab20ca13bddb9bb79ba2cd18fd2998cc751270bcb019004b94c4fe24dcba312
243a108a498288cfb362d5a0e9029a4c3ba11b9cdcdcaaf03c590d0f0a3b602d989092cecb9838ce3858d07626be18623903cf1231fe81e9041f55
368465603d62bf957998c79d0c8b9a8294f11c828d81f4094022716a71d4d766a8518f006dd9affbe2bc38c17408573a983c138238e393c955b
a36cca447c2c8c5468886b941ca9775f617fdef1fd8b34132ed80655ec150036b4df840e0c354f9e6db3ee47fbdbd825aef734fb73bbff0386b3
a255948e2d847a0e4eb1789dbdb8ea7d59e7e4ea0b89ad0bedb539e6f49dd24 (carry: 1)
```

Для перевірки розрахунку використано онлайн-калькулятор багаторозрядної арифметики. Отриманий результат збігається з точністю до розряду:

Обери систему числення

HEX

Введи число A

eecec2cfb3ea7ab2bb0643950e5e829d6125a6eba7c

Введи число B

f1052a6a01d365ca23eb1674bb4c4ae15a0e13c6657

Введи модуль M (якщо потрібно)

Введіть модуль

Обрахувати

A + B =

1dfd3ed39b5bde07cdef15a09c9aacd7e
bb33bab20ca13bddb9bb79ba2cd18fd29
98cc751270bcb019004b94c4fe24dcbafe
312243a108a498288cfb362d5a0e9029a
цcha11b9cdcdaf03c590d0f0a3b607d9

A - B =

-236679a4de8eb1768e4d2dfacedc843f
8e86cdabe4465c4b0ac24bdee47c622ca
07738613e292c76eb0a3dd3d345acfd77
3ba9345d84165af64ff57e31b330c9745
1855f0f0d66c8bf7d71ц79f1ц65c7ц790

3) віднімання чисел;

Віднімання беззнакових чисел реалізовано у методі `sub` за стовпчиковим алгоритмом із послідовною обробкою 64 машинних слів від молодших розрядів до старших. Для коректного врахування можливої позики проміжна різниця обчислюється у знаковому 64-бітному типі. Якщо значення `temp < 0`, відбувається позика з наступного старшого слова. Молодші 32 біти результату фіксуються в масиві `res.digits[i] = static_cast<uint32_t>(temp)`. Метод повертає фінальне значення `borrow` (0 або 1). Нуль сигналізує про коректне віднімання без виходу в негативні числа, а одиниця свідчить про ситуацію $A < B$ (заиклення за модулем 2^{2048}). Оператор `-` виконує цю операцію в кільці лишків за модулем 2^{2048} .

Приклад:

```
a = eecec2cfb3ea7ab2bb0643950e5e829d6125a6eba72e6b0c8487aa7e1f44e4d7e7c2a9e589949c1d10aa0ab78956f97debdeb3d82d983199
3c93fdef251f6dc1c92fd9a5955603371185f73e2b7faeacc730428e4119f88ca6414198555eaaafb24284d495b6b5c61481791b81ad15c537ac3
51a93bb3cbd7ef9a0f6fc4179c41cf2a1513b3f1ceff196dac32035518ff51bab79dcbbc8493285353814549bbaea147de43740a4cef3ee2f7c4d
96286ebaf015d1f4bfcd6d16ab5087be355e58afa40fe7b69f3fb371477eb5515219a8c6fd22089f98db08e423ef8d2d708d11adeeeb15a1515e
404360ee2b2bef60715bc674d7a04b4ad3ead116d61c56f68a07
b = f1052a6a01d365ca23eb1674bb4c4ae15a0e13c66572d0d13533cf3c0d8caafab1ca1d6b9d772ee47f5aae94c68b544dc3526e6b737072fe
ebf8fd47083aa0ce6074f1fb8646d9a39d7dce52a570f5093ba9467aebd2c0f6e6a243f4b203c0e661fb42f395b7c386d678b03d3896ea028912
da5259c5cccfad727c2abe7d54dab3636ce0554e5372510428a56353388faeb32212301a73063c3ed042e5fc81296dbaff5c88b0eb42de74cb75
329cd7cd96a36fd5d7a88901529e981a7de2da3ddc55770ab0c3b7dcb0c22b7202e03e1441c2771f24a7520b4f602a0e4f6326bd4b3a43a7917a
075d83fcec5dec7b1d4c0f29a6ae556dc6e5edc47d821853531d

a - b (hex) = fdc99865b21714e8971b2d20531237bc0717932541bb9a3b4f53db4211b839dd35f88c79ec1d6d38914f5c22c2cba530288c4
56cba27be9a509b00a81ce4ccf368bae7aa0f0f2993740828eb860eb9a38b86fc1355473795bf9efda3a35aea14c22d0a55c5b398da719ee17ae
23a7250f1b07756e1edfee842279345059a47671bc6a8335ea37b8cc869838ca001e06fa307958b9badd62c48f667d16e9f39c0a6c2e4dab7f3e
3b10fbab0d8638b96ed5972621ee824e41558b1efa3b77b7e71c7ba70abee7bfb9496bc89df4f396ab2bb5f91807433b6d8d48f631f2129eaf0a
3b0d1f9bfe438e5dcf13ece02e5540fb74b30f1f5dd0d04e352589a3ea336ea (borrow: 1)
```

Перевірка. Онлайн-калькулятор відобразив результат зі знаком мінус: -236679a4de.... У нашій беззнаковій арифметиці операція виконується в кільці залишків за модулем 2^{2048} :

$$\text{res} = (a - b) \bmod 2^{2048} = 2^{2048} - (b - a)$$

При цьому програма коректно встановила прапорець позики `borrow = 1`.

4) множення чисел, піднесення чисел до квадрату;

Множення двох багаторозрядних чисел реалізовано у методі `mul` за класичним стовпчиковим алгоритмом. Для обчислення повного результату без втрати точності використовується зовнішній буфер `res_words` розміром $2 \times WORDS = 128$ слів (4096 біт). Зовнішній цикл послідовно ітерується по кожному слову другого множника `b_i`: якщо воно ненульове, допоміжний метод `mul_dig` обчислює добуток усього першого числа `A` на це 32-бітне слово, використовуючи 64-бітне множення `uint64_t prod = static_cast<uint64_t>(digits[j]) * val + carry`. Отриманий результат зсувається на `i` слів ліворуч і додається до буфера з урахуванням виникаючого міжсловного переносу. Оператор `*` копіює з буфера лише молодші 64 слова, виконуючи операцію в кільці залишків за модулем 2^{2048} .

Піднесення до квадрата винесено в окремий метод `square`, який приймає на вхід подвійний 128-слівний буфер для збереження повного 4096-бітного значення A^2 . Операція делегується функції стовпчикового множення числа самого на себе.

Приклад:

```
a = eecsec2cfb3ea7ab2bb0643950e5e829d6125a6eba72e6b0c8487aa7e1f44e4d7e7c2a9e589949c1d10aa0ab78956f97debdeb3d82d983199
3c93fdef251f6dc1c92fd9a5955603371185f73e2b7faeacc730428e4119f88ca6a14198555eaaafb24284d495b6b5c61481791b81ad15c537ac3
51a93bb3cbd7ef9a0f6fc4179c41cf2a1513b3f1ceff196dac32035518ff51bab79dcbbc8493285353814549bbaea147de43740a4cef3ee2f7c4d
96286ebaf015d1f4bfcd6d16ab5087be355e58afa40fe7b69f3fb371477eb5515219a8c6fd22089f98db08e423ef8d2d708d11adeeeb15a1515e
404360ee2b2bef60715bc674d7a04b4ad3ead116d61c56f68a07
b = f1052a6a01d365ca23eb1674bb4c4ae15a0e13c66572d0d13533cf3c0d8caafab1ca1dbb9d772ee47f5aae94c68b544dc3526e6b737072fe
ebf8fd47083aa0ce6074f1fb8646d9a39d7dce52a570f5093ba9467aebd2c0f6e6a243f4b203c0e661fb42f395b7c386d678b03d3896ea028912
da5259c5ccfad727c2abe7d54dab3636ce0554e5372510428a56353388faeb32212301a73063c3ed042e5fc81296dbaff5c88b0eb42de74cb75
329cd7cd96a36fd57a88901529e981a7de2da3ddc55770ab0c3b7dcb0c22b7202e03e1441c2771f24a7520b4f602a0e4f6326bd4b3a43a7917a
075d83fcec5dec7b1d4c0f29a6ae556dc6e5edc47d821853531d

a * b (mod) = 63877d2f9ed47dd3e85c76b72eddd3599e9e4ab6eacbb23e6e84edad77b92328d2b3a83199209b4581b34c2c2ae14adf90415
5fa45cd2f24370872d7038f554463e82378963c6c3eda13ca9460a8e25fc9709deec7898a9155d80a0b530b8704f758bc62942a912db94ef04d
949f43427117970c173a26f6b06c40c89e9df7ba479cc5c2e54cfa027aa8b3080a23164712cab2bf2fabb1faf63bd7acbcd35435f81b09b6c73f
c464f13132fd96e9a7964f3cb95bacfb7e6331bf30daff60fbff91bc9528617e9b5ff70678b0a54e7f2f13a29de1df37e089dbe3a4d6868c25d
ed3c019af80777d062c99cedf846b5d792a01e8928a5785dd4c42d530f2ef7c
a * b (full) = e0d5770025b500bd232f1d6e4d00bc42c72adcc216744db0153f7e0a5773082187818fc721829cf449f158bd83a037f338
c142f06b041affa3b001a20ecc05b3070547d83d104e4a6cd2be6e759d58adc7e94122cde24d8126c6ad59802345bcb3ef679c7a810d4698f30c
f889e0ad4a8f03c5c0b7fd790e5488367594909fc2e3587b0e08c0a3501287c913d4c1ff68e023946a58906a1ecceeedcf7b3fe3d4dc0037164e
440af298800214ecfb7133ec1fa90d0aee85bb4e8e3cfb81cef58c75bfff64dbba129f3270d902700a9993f81f01e41e02ef19ae71842f1596601
d8e8945065ea5777a36628f73591326ff45271ae55cc1bc9e8a070f67ba8a763877d2f9ed47dd3e85c76b72eddd3599e9e4ab6eacbb23e6e84ed
ad77b92328d2b3a83199209b4581b34c2c2ae14adfc904155fa45cd2f243740872d7038f554463e82378963c6c3eda13ca9460a8e25fc9709dee
c7898a9155d80a0b530b8704f758bc62942a912db94ef04d949f43427117970c173a26f6b06c40c89e9df7ba479cc5c2e54cfa027aa8b3080a23
164712cab2bf2fabb1faf63bd7acbcd35435f81b09b6c73fc464f13132fd96e9a7964f3cb95bacfb7e6331bf30daff60fbff91bc9528617e9b5
ff70678b0a54e7f2f13a29de1df37e089dbe3a4d6868c25ded3c019af80777d062c99cedf846b5d792a01e8928a5785dd4c42d530f2e7cb
a^2 (full) = dec519384d0cfcb5a0ffbb298584190cbe510c002e74a777f326d5ceb0f0d139d804e1ed178030f0360eddb94427e2b0318e8e
09dc9e14e2a6916334be4cc9765590115382df64e6159bdb2ea0cd4863f0e4b34e1c0cb1a3b4660707f56913da472df15ab618eb20b1217e32c1
eae0c557aa2a1e24de518b6cb237fda7730e8f390d581da34f546c4de5d7c7b23d9e0dada82dcd098e07be89510f4ae69618bf0363bd6d5ef132
52a7087058765bb7c80f8a0cfce210d8908753ab41e4e0bad6e38a8b212befbc7174ae75f36dd0ec69661041dd6d2371f52b01433f452130295
435ca45bb590c9cb336625c6af433dabe73967b93dd98e6a0b81e96c386a44fa6cb01ec262a1436c5a0db29008d33a2563a93724f663428113e
936e17cbe82f1c06451f76b016ca728e7255463e33a39adc6cddb2e8a15df43e45e047a08c65c89d314afc2c3e912f6e14be9ee2c3243b5d918f1
602c8ad343a78d7cd4cc37cbe9e5b802c5912d897e07035d87dec412d94f963ad2861cb0e4facdcb259b5d31b19ec809975851400a709871c460
6ef7289cceb89587433564fe42f801ff80ddd4605610682adf1cf2368c63d4d0bf858cdedcd3907fbd233ab1e7f55073b2960f627b01c523d77
3901f1215a37df84107af8a64817d271d4ebd6fac8167edba48ccce8df53968f692907274e81fab2b74f50e38c312797077b243df8c31
```

Перевірка:



$$A * B =$$

```
e0d5770025b500bd232f1d6e4d00bc42c72adcc216744d54db0153f7e0a5773082187818f
c721829cf449f158bd83a037f338c142f06b041affa3b001a20ecc05b3070547d83d104e4
a6cd2be6e759d58adc7e94122cde24d8126c6ad59802345bcb3ef679c7a810d4698f30cf8
89e0ad4a8f03c5c0b7fd790e5488367594909fc2e3587b0e08c0a3501287c913d4c1ff68e
023946a58906a1eccceedcf7b3fe3d4dc0037164e440af298800214ecfb7133ec1fa90d0a
```



$$A^2$$

```
dec519384d0cfefb5a0fffb298584190cbe510c002e74a777f326d5ceb0f0d139d804e1ed17
8030f0360edbb94427e2b0318e8e09dc9e14e2a6916334be4cc9765590115382df64e6159
bdb2ea0cd4863f0e4b34e1c0cb1a3b4660707f56913da472df15ab618eb20b1217e32c1ea
e0c557aa2a1e24de518b6cb237fda7730e8f390d581da34f546c4de5d7c7b23d9e0dada82
dcd098e07be89510f4ae69618bf0363bd6d5ef13252a7087058765bb7c80f8a0cfce210d8
```

5) ділення чисел, знаходження остачі від ділення;

Цілочисельне ділення та знаходження остачі реалізовано в єдиному методі `div` за алгоритмом зі зсувом і відніманням. Після перевірки ділення на нуль і випадку $A < B$, алгоритм у циклі порівнює поточний залишок R із дільником B . На кожному кроці оцінюється різниця їхніх бітових довжин $\text{shift} = \text{bit_length}(R) - \text{bit_length}(B)$, дільник зсувається ліворуч на цю величину `b.shift_l(shift, b_shifted)`, після чого віднімається від залишку `r.sub(b_shifted, r)`, а відповідний біт частки встановлюється додаванням `1 << shift`. Оператори `/` та `%` звертаються до методу `div`, повертаючи сформовану частку Q або фінальну остачу R .

Приклад:

```
a = eec2cfb3ea7ab2bb0643950e5e829d6125a6eba72e6b0c8487aa7e1f44e4d7e7c2a9e589949c1d10aa0ab78956f97debdeb3d82d983199
3c93fdef251f6dc1c92fd9a5955603371185f73e2b7faeacc730428e4119f88ca6414198555eaaafb24284d495b6b5c61481791b81ad15c537ac3
51a93bb3cbd7ef9a0f6fc4179c41cf2a1513b3f1ceff196dac32035518ff51bab79dc9c8493285353814549bbaea147de43740a4cef3ee2f7c4d
96286ebaf015d1f4bfc6d16ab5087be355e58afa40fe7b69f3fb371477eb5515219a8c6fd22089f98db08e423ef8d2d708d11adeeb15a1515e
404360ee2b2bef60715bc674d7a04b4ad3ead116d61c56f68a07
b = f1052a6a01d365ca23eb1674bb4c4ae15a0e13c66572d0d13533cf3c0d8caafab1ca1d6b9d772ee47f5aae94c68b544dc3526e6b737072fe
ebf8fd47083aa0ce6074f1fb8646d9a39d7dce52a570f5093ba9467aebd2c0f6e6a243f4b203c0e661fb42f395b7c386d678b03d3896ea028912
da5259c5cccfad727c2abe7d54dab3636ce0554e5372510428a56335388faeb32212301a73063c3ed042e5fc81296dbaff5c88b0eb42de74cb75
329cd7cd96a36fd5d7a88901529e981a7de2da3ddc55770ab0c3b7dcb0c22b7202e03e1441c2771f24a7520b4f602a0e4f6326bd4b3a43a7917a
075d83fcec5dec7b1d4c0f29a6ae556dc6e5edc47d821853531d
```

```
a / b (hex) = 0
a mod b (hex) = eec2cfb3ea7ab2bb0643950e5e829d6125a6eba72e6b0c8487aa7e1f44e4d7e7c2a9e589949c1d10aa0ab78956f97debde
b3d82d9831993c93fdef251f6dc1c92fd9a5955603371185f73e2b7faeacc730428e4119f88ca6414198555eaaafb24284d495b6b5c61481791b8
1ad15c537ac351a93bb3cbd7ef9a0f6fc4179c41cf2a1513b3f1ceff196dac32035518ff51bab79dc9c8493285353814549bbaea147de43740a4
cef3ee2f7c4d96286ebaf015d1f4bfc6d16ab5087be355e58afa40fe7b69f3fb371477eb5515219a8c6fd22089f98db08e423ef8d2d708d11ad
eeeb15a1515e404360ee2b2bef60715bc674d7a04b4ad3ead116d61c56f68a07
```

Перевірка:



A / B =

0



A mod B =

```
eecec2cfb3ea7ab2bb0643950e5e829d6125a6eba72e6b0c8487aa7e1f44e4d7e7c2a9e58
9949c1d10aa0ab78956f97debdeb3d82d9831993c93fdef251f6dc1c92fd9a59556033711
85f73e2b7faeacc730428e4119f88ca6414198555eaa7b24284d495b6b5c61481791b81ad
15c537ac351a93bb3cbd7ef9a0f6fc4179c41cf2a1513b3f1ceff196dac32035518ff51ba
b79dcb8c8493285353814549bbaea147de43740a4cef3ee2f7c4d96286ebaf015d1f4bfc6
```

б) піднесення числа до багаторозрядного степеня;

Піднесення до степеня реалізовано у методі `BigInt::power(a, b, c)` за схемою Горнера зліва направо за модулем 2^{2048} . Змінна `C` ініціалізується одиницею `BigInt::one()`, після чого обчислюється бітова довжина показника степеня (якщо $m = 0$, повертається 1, що відповідає $A^0 = 1$). Основний цикл ітерується від старшого значущого біта показника степеня ($m - 1$) до молодшого. Якщо поточний біт дорівнює одиниці (`b.get_bit(i) == 1`), акумулятор множиться на основу `c.mul(a, buf)` із збереженням молодших 64 слів у `C`, а для всіх кроків, крім останнього, результат підноситься до квадрата `c.square(buf)` за модулем 2^{2048} .

Приклад:

```
a^0 (hex) = 1
a^1 (hex) = eecec2cfb3ea7ab2bb0643950e5e829d6125a6eba72e6b0c8487aa7e1f44e4d7e7c2a9e589949c1d10aa0ab78956f97debdeb3d8
2d9831993c93fdef251f6dc1c92fd9a5955603371185f73e2b7faeacc730428e4119f88ca6414198555eaa7b24284d495b6b5c61481791b81ad1
5c537ac351a93bb3cbd7ef9a0f6fc4179c41cf2a1513b3f1ceff196dac32035518ff51bab79dcb8c8493285353814549bbaea147de43740a4cef3
ee2f7c4d96286ebaf015d1f4bfc6d16ab5087be355e58afa40fe7b69f3fb371477eb5515219a8c6fd22089f98db08e423ef8d2d708d11adeeeb
15a1515e404360ee2b2bef60715bc674d7a04b4ad3ead116d61c56f68a07
2^10 (hex) = 400
```

7) конвертування (переведення) числа в символьну строку та обернене перетворення символьної строки у число; обов'язкова підтримка шістнадцяткового представлення.

Перетворення шістнадцяткового рядка у число реалізовано у статичному методі `from_hex`, який ігнорує необов'язковий префікс `0x`, валідує коректність

hex-символів через `std::isxdigit` та парсить рядок з кінця блоками довжиною до 8 символів, записуючи їх у масив `digits` від молодших розрядів до старших. Зворотнє перетворення числа у шістнадцятковий рядок реалізовано у методах `to_hex` та `to_hex_adv`. Алгоритм знаходить індекс старшого ненульового слова (зануляючи провідні нулі для всього числа), форматує його без зайвих нулів через специфікатор `"%x"`, а всі молодші слова послідовно конкатенує з доповненням до 8 позицій через `"%08x"`, повертаючи канонічний hex-рядок без провідних нулів.

Бажано реалізувати такі операції:

- 1) визначення номеру старшого ненульового біта числа;

Визначення бітової довжини (фактично позиції старшого значущого біта плюс 1) реалізовано у методі `bit_length()`. Спочатку алгоритм шукає індекс старшого ненульового 32-бітного слова, рухаючись від кінця масиву. Якщо всі слова виявляються нульовими, метод повертає 0. Для знайденого старшого ненульового слова `top = digits[i]` визначається кількість значущих бітів за допомогою циклічного правого бітового зсуву `top >>= 1` з інкрементом лічильника `bits_in_top`. Підсумковий номер біта розраховується за формулою `static_cast<size_t>(i) * 32 + bits_in_top`.

- 2) бітові зсуви (вправо та вліво), які відповідають діленню та множенню на степені двійки

Побітові зсуви на довільну кількість біт `k` реалізовано у методах `shift_l` (лівий зсув, множення на 2^k) та `shift_r` (правий зсув, ділення на 2^k). Величина зсуву розкладається на зсув за цілими машинними словами `word_shift = k / 32` та побітовий внутрішньословний зсув `bit_shift = k % 32`. Якщо `bit_shift == 0`, слова просто переміщуються на нові індекси з очищенням звільнених позицій. Якщо ж `bit_shift != 0`, кожне 32-бітне слово розщеплюється між двома сусідніми позиціями масиву. Для лівого зсуву молодша частина слова зсувається вгору `digits[i] << bit_shift`, а старші біти переносяться у наступне слово через `digits[i] >> (32 - bit_shift)`; для правого зсуву – навпаки, зі склеюванням бітових масок у бік молодших розрядів. Оператори `<<`, `>>`, `<<=`, `>>=` забезпечують роботу зі зсувами в рамках фіксованої 2048-бітної розрядної сітки.

Приклад:

```
1 << 2047 bit_length = 2048
1 << 2048 = 0
1 << 32 = 100000000
100000000 >> 32 = 1
```

Б) Проконтролювати коректність реалізації алгоритмів; наприклад, для декількох багаторозрядних a, b, c, n перевірити тотожності:

- 1) Дистрибутивність множення відносно додавання та комутативність:

$$(A + B) \cdot C = C \cdot (A + B) = (A \cdot C) + (B \cdot C) \bmod 2^{2048}$$

- 2) Множення як багаторазове додавання ($n \geq 100$):

$$n \cdot A = A + A + \dots + A \bmod 2^{2048}$$

3) Інваріант цілочисельного ділення:

$$A = B \cdot Q + R$$

4) Формула різниці квадратів:

$$(A + B)(A - B) = A^2 - B^2 \bmod 2^{2048}$$

5) Мультиплікативна властивість показників степеня:

$$A^{x+y} = A^x \cdot A^y \bmod 2^{2048}$$

6) Оберненість операцій побітового зсуву:

$$(A \ll k) \gg k = A$$

7) Еквівалентність ділення на степінь двійки та правого зсуву:

$$\frac{A}{2^k} = A \gg k$$

```
./test
a = d9efdbb753ea6d784b1aefd808629858e02825dbaa7bb80f1ebfc8cb6bd8c7b377f317bd5784c05e92a792b607ed40124c0d1633a476a13
65f3e757da0c2c3afbb08410b15ae7046e08f776d51ee30b9b907d4359163fa70f619bd4ea36b5dc9c8550b1371484037eb9b4cab1edada9ee
c6697f0d7e638643ad2a5e47690b26326dbde7f7ebe847f3bb9ea7e4561ddd02e72f1208603eb78194d5384deb1e71a9dc887b56392f00cd776
eac3a0ac75a163f7262d5f3de4dd42cf168ce988d398dcd3ab135f7c7afab34af4f52a4fa5a61311efcea8f657d7ddf668b6fc33c174c66a38d9
deb3a28bc551a693cf2a637790f91a1a6032f1e280e95f3d36ad

b = 742b7f3726e9b7c73559552cedafbb89d0bfff2c07ab1006f7d36fdc27dc52616dc46751a2abfd91e1b7e7375c8db92dc6569477fef0900b6
2ca31d1ae8fc6aacff8e8c0da5cc8bd5cb030e39b68698035f075995e7813bf610c257b5edd9c953ad27dcf94ba6389f3a820b7a374539c6cfff5d
6fe9c66b37ab9c84ac259886e7c3f97bea646334e05592dc08bb64279861cfd8598a477f007defa719bc864b9bc9f29040a608a26ee4543f4421
018e432a8fbaa0179aa262accf7a3c09421f5fc310df3bd1eed5560ce60f8230ae261f73fb9c926fd3400f1ae734bcc33e11606a67ed06403ff3
b6691467d070c8c0ff45fd6dd9363c56e5b7ae6b30d5223c03ec

c = 9b363f945a3af40d2ac088cf16cb050e28b8426bcb85d78fc996c35380e7bbce93a740332be966f8340c3eaa1ead89600415724f8844f393
a1a339f584104cce95457ea4eef599bdc1f99b1b74aad7ee3775fc21855bd86a8e0832db72374c2f77dddbeccc5b240932268859b2ff127d0ce2
d42fb59cd66db230833d347f44fa1a9f1dff0c0570958df42cd7c82607c2d5e80f22891316e0be183cc5b6880381271d20ad302f33169144b985
9870c4dce8edb0f84ef118bffc0e1ac775b62b6ca99757a93ed4ba7c5d4256be16ba163b8b4123e80347eb1b835ca10eee31b3c41998d3706e19
4de7aaa92feead0cc5cc9cc9e05d3357888afd291b431db40dd6
(a + b) * c = a2f40d4944987441b5858a6d28a3d6e80897838b377baa87432ad41031701aba9aeb4b2c7b1cd0f8c98d30b271a9b742
b0b8296da65d08565fb3602d1a7dca847151003ce1da41e5f2b458e10317cb2906f44c3c9682aabd12a38617c495172f9537f40881b63a33677
5574f3b1a05d8ae5395401945238b8b53bc6c90b77dd5d15877be1a1ef0f1fd362eaa2d959783799a7a6bb27a488cf546807393e1e365b40dfe5
a2574f618dfbd33a5ff41464b4f506d5c47722854f3df950c58ddb7dd9db4985f561f9a290bdf3e7e73d612295ef9a0e2196b9d290344a2f4ded
56ed84a76b02c326afbbcfb70c1e652bfc96889abd663b64e65ac5007fbeebe4c0e6
(a * b) + (b * c) = a2f40d4944987441b5858a6d28a3d6e80897838b377baa87432ad41031701aba9aeb4b2c7b1cd0f8c98d30b271a9b742
b0b8296da65d08565fb3602d1a7dca847151003ce1da41e5f2b458e10317cb2906f44c3c9682aabd12a38617c495172f9537f40881b63a33677
5574f3b1a05d8ae5395401945238b8b53bc6c90b77dd5d15877be1a1ef0f1fd362eaa2d959783799a7a6bb27a488cf546807393e1e365b40dfe5
a2574f618dfbd33a5ff41464b4f506d5c47722854f3df950c58ddb7dd9db4985f561f9a290bdf3e7e73d612295ef9a0e2196b9d290344a2f4ded
56ed84a76b02c326afbbcfb70c1e652bfc96889abd663b64e65ac5007fbeebe4c0e6
Distributivity test passed!

1000 * a = 50f2441fcbbb9de8a8138e3e0c1231b2b9cd3e211f346fb101d285a9d46cc150c8d94bb9dde8f716cde95170ef6c24779131eb9ba
6f65bc640baf2bbf8c8cc6672983e134cb16694dd306a83080a6e557ac694f13fee8a4941548b7b2e5b665c368c3343f2823ada7056a37c7086
e5fccb70c184cb5b4c4785c6d8046f2538d34fcadd2207183591014e3bdf3f064aaf23570fde70b7f4fcd22d60e3f05e6eebfff857521c8cf5f9b
229a8850c2ba1ab7e6e7d6d213c09c6004cf8f016703e7a8d1e9ad443acfe40634c5cccdad471f10ba7e10bf440247233b0a890ac92a2bb0270
eee130dedb2f1fac6f2b1613d948b0e4d0df707c700dcc78f9c071d93c8
sum = 50f2441fcbbb9de8a8138e3e0c1231b2b9cd3e211f346fb101d285a9d46cc150c8d94bb9dde8f716cde95170ef6c24779131eb9ba6f65b
c640bfaf2bbf8c8cc6672983e134cb16694dd306a83080a6e557ac694f13fee8a4941548b7b2e5b665c368c3343f2823ada7056a37c7086e5fcc
b70c184cb5b4c4785c6d8046f2538d34fcadd2207183591014e3bdf3f064aaf23570fde70b7f4fcd22d60e3f05e6eebfff857521c8cf5f9b229a8
850c2ba1ab7e6e7d6d213c09c6004cf8f016703e7a8d1e9ad443acfe40634c5cccdad471f10ba7e10bf440247233b0a890ac92a2bb0270eee13
0dedb2f1fac6f2b1613d948b0e4d0df707c700dcc78f9c071d93c8
Multiple addition test passed!
```

```

q = 1
r = 65c45c802d00b5b1169859d092d66dfb428f9d3ff6bb1174b4feca38f866645b38bc61aab872e7cdac05b597a34124bf5789e3
4b3e695d3950ca3cf10fc17f3b12efa9b3e5cd8a543d255c0e6b9ead1b25f71ecb4556de9ae9e63cf74da17b0a1eb5bbf6d38a905b5e0211565e
5140aa91567fb8a246b7e9bf0104c5c081472cb6835984c30b92b517b2e343bcdbdc0e17d4e8a9a18585fbd0ff90cd3942e7f48a5d227f12f4ae
9bcd9355e9355d81e5e6c3df8b8af9c1156306c5d46d89c5c2b9a101bc3e096f94eb311a46cf0adbaa0980a21c8e99db70a321332aa59bc95987
c029f8e6284a8e23f4e0dd2cfe46609b7c2ddc37a7b437750143d0132c1
b * q + r = d9fedbb735ae6d784bf1aef808629858e02825dbaa7bb80f1ebfc8cb6bd8c7b377f317bd5784c05e92a792b607ed40124c0d163
3a476a1365f3e757da0c2c2c3afbb08410b15ae7046e08f776d51ee30b9b907d4359163fa70f619bd4ea36b5dc9c8550b1371484037eb9b4cab1
edada9eec6697f0d7e638643ad2a5e47690b2632dbde7f7ebe847f3bb9ea7e4561ddd0f02e72f1208603eb78194d5384deb1e71a9dc887b56392
f00cd776eac3a0ac75a163f7262d5f3de4dd42cf168ce988d398dcd3ab135f7c7afab34af4f52a4fa5a61311efcea8f657d7dd668b6fc33c174
c66a38d9deb3a28bc551a693cf2a637790f91a1a6032f1e280e95f3d36ad
Division invariant test passed!

(a + b) * (a - b) = 8a209b0b71c000e1ded31840e65b07025c9a20d34c7054d67c472c29e365136855bfc78858a80a6c90617bac21bf394c
b910476b52c01a49330857039c9cc4eae408c8a59126f03006558ccd1aa8a28c8efaa2375f42f3e4c9d1e08f899d93a476a3c795415da8f1a516
8c696fa4b2b1fd5ab209c068c84086989af78923461e4c663b491690354596ca2ab6566e074803058714c8d8dadacddf2522a0107e6c69a4188b
41e05186dbbb81f5ff5b83a7240dd47d463e16b8f74c092dca6b0cc28d437eab76a13dc662b08ee94519c755f3011c7878ab7230d1cd18a94b7c
1263b750187fc299b7b49f2cab231272c40e4e4080d774f0136565c9f9e4cf9700f59
a^2 - b^2 = 8a209b0b71c000e1ded31840e65b07025c9a20d34c7054d67c472c29e365136855bfc78858a80a6c90617bac21bf394cb910476b
52c01a49330857039c9cc4eae408c8a59126f03006558ccd1aa8a28c8efaa2375f42f3e4c9d1e08f899d93a476a3c795415da8f1a5168c696fa4
b2b1fd5ab209c068c84086989af78923461e4c663b491690354596ca2ab6566e074803058714c8d8dadacddf2522a0107e6c69a4188b41e05186
dbbb81f5ff5b83a7240dd47d463e16b8f74c092dca6b0cc28d437eab76a13dc662b08ee94519c755f3011c7878ab7230d1cd18a94b7c1263b750
187fc299b7b49f2cab231272c40e4e4080d774f0136565c9f9e4cf9700f59
Difference of squares test passed!

a^3 * a^4 = b3d4ebd71004caba363f395a5f4f23034aab5fb0edbb49d8cc2b824f38b6f7d0ffad6f18e6e42812a89dc4548f614f4cdc7a2e6
a4765ac88a4a7d99c0484e8ec588eef4d10ab2c019f1fc5653d244c7269380bc92de6a5560985cf9291722ee8b003cba15c0a29384b7cdd3b1a7
71543056c53caf2cad87c14b28593b9827d88473c863efaba2017317546f2a89182d02950d469144c4175b0e32e19bc88c11e3db679121f36db
2ac2d53aece6a3bbdc6634279c8a729000f5878eedff14570579d04c028ea5ea5ae7211053149e9c9db6f045d9ab77e07b5de31961df194cce85d
a4045c8c91f12eb632deab21b04539e40c8476efacc85949cca23e9ceec5
a^7 = b3d4ebd71004caba363f395a5f4f23034aab5fb0edbb49d8cc2b824f38b6f7d0ffad6f18e6e42812a89dc4548f614f4cdc7a2e6a4765a
c88a4a7d99c0484e8ec588eef4d10ab2c019f1fc5653d244c7269380bc92de6a5560985cf9291722ee8b003cba15c0a29384b7cdd3b1a7715430
56c53caf2cad87c14b28593b9827d88473c863efaba2017317546f2a89182d02950d469144c4175b0e32e19bc88c11e3db679121f36db2ac2d5
3aece6a3bbdc6634279c8a729000f5878eedff14570579d04c028ea5ea5ae7211053149e9c9db6f045d9ab77e07b5de31961df194cce85da4045c
8c91f12eb632deab21b04539e40c8476efacc85949cca23e9ceec5
Power rule test passed!

a_masked = 8e02825dbaa7bb80f1ebfc8cb6bd8c7b377f317bd5784c05e92a792b607ed40124c0d1633a476a1365f3e757da0c2c2c3afbb0841
0b15ae7046e08f776d51ee30b9b907d4359163fa70f619bd4ea36b5dc9c8550b1371484037eb9b4cab1edada9eec6697f0d7e638643ad2a5e476
90b26326dbde7f7ebe847f3bb9ea7e4561ddd0f02e72f1208603eb78194d5384deb1e71a9dc887b56392f00cd776eac3a0ac75a163f7262d5f3de
4dd42cf168ce988d398dcd3ab135f7c7afab34af4f52a4fa5a61311efcea8f657d7dd668b6fc33c174c66a38d9deb3a28bc551a693cf2a63779
0f91a1a6032f1e280e95f3d36ad
(a_sh << 100) >> 100 = 8e02825dbaa7bb80f1ebfc8cb6bd8c7b377f317bd5784c05e92a792b607ed40124c0d1633a476a1365f3e757da0c2
c2c3afbb08410b15ae7046e08f776d51ee30b9b907d4359163fa70f619bd4ea36b5dc9c8550b1371484037eb9b4cab1edada9eec6697f0d7e638
643ad2a5e47690b26326dbde7f7ebe847f3bb9ea7e4561ddd0f02e72f1208603eb78194d5384deb1e71a9dc887b56392f00cd776eac3a0ac75a16
3f7262d5f3de4dd42cf168ce988d398dcd3ab135f7c7afab34af4f52a4fa5a61311efcea8f657d7dd668b6fc33c174c66a38d9deb3a28bc551a
693cf2a637790f91a1a6032f1e280e95f3d36ad
Shift roundtrip test passed!

a / (1 << 77) = 6cf7eddba9f536bc25f8d77ec04314c2c701412edd53ddc078f5fe465b5ec63d9bbf98bdeabc2602f4953c95b03f6a009260
68b19d23b509b2f9f3abed06161d7dd8420858ad738237047bbb6a8f7185cdc83ea1ac8b1fd387b0cdea751b5aee4e42a8589b8a4201bf5cda
6558f6d6d4f76334bf86bf31c321d6952f23b485931936def3fbf5f423f9ddcf53f22b0eeef8173978904301f5bc0ca6a9c26f58f38d4ee443da
b1c978066bbb7561d0563ad0b1fb9316af9ef26ea1678b4674c469cc6e69d589afbe3d7d59a57a7a9527d2d30988f7e7547b2bebeefb345b7e19
e0ba63351c6cef59d145e2a8d349e79531bbc87c8d0d3
a >> 77 = 6cf7eddba9f536bc25f8d77ec04314c2c701412edd53ddc078f5fe465b5ec63d9bbf98bdeabc2602f4953c95b03f6a00926068b19d
23b509b2f9f3abed06161d7dd8420858ad738237047bbb6a8f7185cdc83ea1ac8b1fd387b0cdea751b5aee4e42a8589b8a4201bf5cda6558f6
d6d4f76334bf86bf31c321d6952f23b485931936def3fbf5f423f9ddcf53f22b0eeef8173978904301f5bc0ca6a9c26f58f38d4ee443dab1c978
066bbb7561d0563ad0b1fb9316af9ef26ea1678b4674c469cc6e69d589afbe3d7d59a57a7a9527d2d30988f7e7547b2bebeefb345b7e19e0ba63
351c6cef59d145e2a8d349e79531bbc87c8d0d3
Div by 2^k vs shift test passed!

```

В) Обчислити середній час виконання реалізованих арифметичних операцій. Підрахувати кількість тактів процесора (або інших одиниць виміру часу) на кожну операцію. Результати подати у вигляді таблиць або діаграм.

Для оцінки обчислювальної складності розробленої бібліотеки довгої арифметики було проведено серію бенчмарків у середовищі Kali Linux. Вимірювання кількості процесорних тактів здійснювалося апаратно за допомогою інструкції `__rdtsc()`, а фізичного часу виконання – за допомогою таймера високої точності `std::chrono::high_resolution_clock`. Дослідження проводилося для двох різних конфігурацій. Конфігурація 1 моделює загальний випадок із суттєвою різницею розрядностей при діленні (2048 біт на 1024 біти) та 2048-бітним показником степеня; Конфігурація 2 відображає ділення двох повнорозрядних 2048-бітних чисел (де частка має малий розмір) та піднесення до невеликого показника степеня ($e = 17$).

```
(kali@kali)-[~/Desktop]
$ ./bench
Add (+): 696 cycles | 0.211528 us
Sub (-): 762 cycles | 0.231494 us
Shift (<<): 1639 cycles | 0.497717 us
Mul (*): 104903 cycles | 31.8487 us
Sqr: 113521 cycles | 34.4654 us
Div: 4475584 cycles | 1358.79 us
Power: 334626602 cycles | 101593 us
```

```
(kali@kali)-[~/Desktop]
$ ./bench
Add (+): 688 cycles | 0.208982 us
Sub (-): 785 cycles | 0.238324 us
Shift (<<): 1682 cycles | 0.510864 us
Mul (*): 102139 cycles | 31.0093 us
Sqr: 112728 cycles | 34.2242 us
Div: 6534 cycles | 1.9819 us
Power: 794318 cycles | 241.152 us
```

Таблиця результатів вимірювання

Операція	Такти (Конф. 1)	Час, мкс (Конф. 1)	Такти (Конф. 2)	Час, мкс (Конф. 2)
Додавання (+)	696	0.212	688	0.209
Віднімання (-)	762	0.231	785	0.238
Побітовий зсув (<< 37)	1639	0.498	1682	0.511
Множення (*)	104903	31.85	102139	31.01
Піднесення до квадрата (sqr)	113521	34.47	112728	34.22
Ділення (div)	4475584	1358.79	6534	1.98
Піднесення до степеня (pow)	334626602	101593.00	794318	241.15

Отримані експериментальні показники часу та кількості процесорних тактів майже узгоджуються з теоретичною складністю відповідних алгоритмів.

Операції додавання та віднімання мають лінійну складність $O(N)$ і реалізуються за один послідовний прохід з урахуванням біта переносу чи позики. Побітовий зсув також належить до класу $O(N)$, проте вимагає комбінування суміжних 32-бітних слів за допомогою бітових масок, що збільшує час виконання приблизно вдвічі порівняно з додаванням.

Алгоритми класичного стовпчикowego множення та піднесення до квадрата характеризуються квадратичною складністю $O(N^2)$. Обчислення повного 4096-бітного добутку потребує виконання подвійного вкладеного циклу на $N \times N = 4096$ операцій базового 64-бітного множення та зберігання результату. Оскільки операція піднесення до квадрата реалізована як множення числа самого на себе, її часові характеристики практично ідентичні добутку двох довільних операндів.

Цілочисельне ділення має складність $O((k_A - k_B + 1) * N)$, де k_A та k_B – фактичні бітові довжини діленого та дільника. Кожна ітерація різниці бітових довжин вимагає зсуву та віднімання довгого числа складності $O(N)$. Цим пояснюється суттєва різниця між замірами. Ділення чисел однакової розрядності завершується швидко, оскільки частка формується в межах однієї-двох ітерацій.

Складність алгоритму піднесення до степеня за схемою Горнера становить $O(k_E * N^2)$, де k_E – бітова довжина показника степеня. На кожному біті виконується піднесення проміжного результату до квадрата, а за наявності одиничного біта – додаткове множення на основу.

Додаток А. Заголовний файл BigInt.hpp

```
#pragma once

#include <iostream>
#include <cstdint>
#include <string>
#include <cstdio>
#include <cctype>
#include <stdexcept>
#include <random>

constexpr size_t WORDS = 64;

struct BigInt {
    uint32_t digits[WORDS];

    BigInt() {
        for (size_t i=0; i < WORDS; ++i) {
            digits[i] = 0;
        }
    }

    BigInt(uint32_t val) {
        digits[0] = val;
        for (size_t i=1; i< WORDS; ++i) {
            digits[i]=0;
        }
    }

    static BigInt zero() {
        return BigInt(0);
    }

    static BigInt one() {
        return BigInt(1);
    }

    static std::string to_hex_adv(const uint32_t* words, size_t n) {
        int i = static_cast<int>(n) - 1;
        while (i > 0 && words[i]==0) {
            --i;
        }

        char buf[16];
        std::snprintf(buf, sizeof(buf), "%x", words[i]);
        std::string s = buf;

        for (int j = i-1; j>= 0; --j) {
            std::snprintf(buf, sizeof(buf), "%08x", words[j]);
            s += buf;
        }
    }
}
```

```

        return s;
    }

    std::string to_hex() const {
        return to_hex_adv(digits, WORDS);
    }

    static BigInt from_hex(std::string hex) {
        BigInt res;

        if (hex.size() >= 2 && hex[0] == '0' && (hex[1] == 'x' || hex[1] ==
symbol")) {
            hex = hex.substr(2);
        }

        for (char c: hex) {
            if (!std::isxdigit(static_cast<unsigned char>(c))) {
                throw std::invalid_argument("incorrect hex
symbol");
            }
        }

        int len = static_cast<int>(hex.length());
        size_t idx = 0;

        for (int i=len; i>0; i-=8) {
            if (idx>= WORDS) {
                throw std::overflow_error("number >= 2048 bits");
            }

            int start = (i>= 8) ? i-8: 0;
            int count = i - start;
            std::string part = hex.substr(start, count);

            res.digits[idx] = static_cast<uint32_t>(std::stoul(part,
nullptr, 16));
            idx++;
        }

        return res;
    }

    uint32_t add(const BigInt& num, BigInt& res) const{
        uint64_t carry = 0;

        for (size_t i = 0; i < WORDS; ++i) {
            uint64_t temp = static_cast<uint64_t>(digits[i]) +
num.digits[i] + carry;
            res.digits[i] = static_cast<uint32_t>(temp);
            carry = temp >> 32;
        }

        return static_cast<uint32_t>(carry);
    }

```

```

    }

    uint32_t sub(const BigInt& num, BigInt& res) const{
        int64_t borrow = 0;

        for (size_t i = 0; i < WORDS; ++i) {
            int64_t temp = static_cast<int64_t>(digits[i]) -
static_cast<int64_t>(num.digits[i]) - borrow;

            if (temp < 0) {
                temp += (1LL << 32);
                borrow = 1;
            } else {
                borrow = 0;
            }

            res.digits[i] = static_cast<uint32_t>(temp);
        }

        return static_cast<uint32_t>(borrow);
    }

    int cmp(const BigInt& num) const {
        for (int i = static_cast<int>(WORDS) - 1; i>=0; --i) {
            if (digits[i] > num.digits[i]) {
                return 1;
            }
            if (digits[i] < num.digits[i]) {
                return -1;
            }
        }

        return 0;
    }

    uint32_t mul_dig(uint32_t val, BigInt& res) const {
        uint64_t carry = 0;

        for (size_t i = 0; i < WORDS; i++) {
            uint64_t prod = static_cast<uint64_t>(digits[i]) * val +
carry;

            res.digits[i] = static_cast<uint32_t>(prod);
            carry = prod >> 32;
        }

        return static_cast<uint32_t>(carry);
    }

    void mul(const BigInt& num, uint32_t res_words[2*WORDS]) const {
        for(size_t k = 0; k < 2*WORDS; ++k) {
            res_words[k] = 0;
        }
    }

```

```

        for (size_t i = 0; i < WORDS; ++i) {
            if (num.digits[i] == 0) continue;

            BigInt temp;
            uint32_t carry_dig = mul_dig(num.digits[i], temp);

            uint64_t carry = 0;
            for (size_t j = 0; j < WORDS; ++j) {
                uint64_t sum =
static_cast<uint64_t>(res_words[i+j]) + static_cast<uint64_t>(temp.digits[j]) +
carry;

                res_words[i+j] = static_cast<uint32_t>(sum);
                carry = sum >> 32;
            }

            res_words[i+WORDS] = static_cast<uint32_t>(carry +
carry_dig);
        }
    }

    void square(uint32_t res_words[2 * WORDS]) const {
        mul(*this, res_words);
    }

    size_t bit_length() const {
        int i = static_cast<int>(WORDS) - 1;
        while (i >= 0 && digits[i] == 0) {
            --i;
        }

        if (i < 0) {
            return 0;
        }

        uint32_t top = digits[i];
        size_t bits_in_top = 0;
        while (top > 0) {
            ++bits_in_top;
            top >>= 1;
        }

        return static_cast<size_t>(i)*32 + bits_in_top;
    }

    void shift_l(size_t k, BigInt& res) const {
        res = BigInt::zero();

        if (k >= WORDS * 32) {
            return;
        }

        size_t word_shift = k / 32;

```

```

        size_t bit_shift = k % 32;

        if (bit_shift == 0) {
            for (size_t i = 0; i + word_shift < WORDS; ++i) {
                res.digits[i + word_shift] = digits[i];
            }
        } else {
            for (size_t i = 0; i + word_shift < WORDS; ++i) {
                res.digits[i + word_shift] |= (digits[i] <<
bit_shift);

                if (i + word_shift + 1 < WORDS) {
                    res.digits[i + word_shift + 1] |=
(digits[i] >> (32 - bit_shift));
                }
            }
        }
    }

    void shift_r(size_t k, BigInt& res) const {
        res = BigInt::zero();

        if (k >= WORDS * 32) {
            return;
        }

        size_t word_shift = k / 32;
        size_t bit_shift = k % 32;

        if (bit_shift == 0) {
            for (size_t i = word_shift; i < WORDS; ++i) {
                res.digits[i - word_shift] = digits[i];
            }
        } else {
            for (size_t i = word_shift; i < WORDS; ++i) {
                res.digits[i - word_shift] |= (digits[i] >>
bit_shift);

                if (i + 1 < WORDS) {
                    res.digits[i - word_shift] |= (digits[i+1]
<< (32 - bit_shift));
                }
            }
        }
    }

    void div(const BigInt& b, BigInt& q, BigInt& r) const {
        if (b.bit_length() == 0) {
            throw std::invalid_argument("Division by zero");
        }

        q = BigInt::zero();
        r = *this;

```

```

        if (r.cmp(b) < 0) {
            return;
        }

        BigInt one = BigInt::one();
        size_t k = b.bit_length();

        while (r.cmp(b) >= 0) {
            size_t t = r.bit_length();
            size_t shift = t - k;

            BigInt b_shifted;
            b.shift_l(shift, b_shifted);

            if (r.cmp(b_shifted) < 0) {
                --shift;
                b.shift_l(shift, b_shifted);
            }

            BigInt next_r;
            r.sub(b_shifted, next_r);
            r = next_r;

            BigInt q_term;
            one.shift_l(shift, q_term);
            BigInt next_q;
            q.add(q_term, next_q);
            q = next_q;
        }
    }

    uint32_t get_bit(size_t idx) const {
        if (idx >= WORDS * 32) {
            return 0;
        }

        size_t word_idx = idx / 32;
        size_t bit_pos = idx % 32;

        return (digits[word_idx] >> bit_pos) & 1U;
    }

    static void power(const BigInt& a, const BigInt& b, BigInt& c) {
        c = BigInt::one();
        size_t m = b.bit_length();

        if (m == 0) {
            return;
        }

        for (int i = static_cast<int>(m) - 1; i >= 0; --i) {
            if (b.get_bit(i) == 1) {

```

```

        uint32_t buf[2 * WORDS];
        c.mul(a, buf);

        for (size_t w = 0; w < WORDS; ++w) {
            c.digits[w] = buf[w];
        }
    }

    if (i != 0) {
        uint32_t buf[2 * WORDS];
        c.square(buf);

        for (size_t w = 0; w < WORDS; ++w) {
            c.digits[w] = buf[w];
        }
    }
}

static BigInt random() {
    static std::random_device rd;
    static std::mt19937_64 gen(rd());
    std::uniform_int_distribution<uint32_t> dis(0, 0xFFFFFFFF);

    BigInt res;
    for (size_t i = 0; i < WORDS; ++i) {
        res.digits[i] = dis(gen);
    }

    return res;
}

BigInt& operator+=(const BigInt& num) {
    BigInt res;
    add(num, res);
    *this = res;
    return *this;
}

BigInt& operator-=(const BigInt& num) {
    BigInt res;
    sub(num, res);
    *this = res;
    return *this;
}

BigInt& operator*=(const BigInt& num) {
    uint32_t buf[2 * WORDS];
    mul(num, buf);

    for (size_t i = 0; i < WORDS; ++i) {
        digits[i] = buf[i];
    }
}

```

```

        return *this;
    }

    BigInt& operator<<=(size_t k) {
        BigInt res;
        shift_l(k, res);
        *this = res;
        return *this;
    }

    BigInt& operator>>=(size_t k) {
        BigInt res;
        shift_r(k, res);
        *this = res;
        return *this;
    }
};

inline BigInt operator+(BigInt num1, const BigInt& num2) {
    num1 += num2;
    return num1;
}

inline BigInt operator-(BigInt num1, const BigInt& num2) {
    num1 -= num2;
    return num1;
}

inline BigInt operator*(BigInt num1, const BigInt& num2) {
    num1 *= num2;
    return num1;
}

inline BigInt operator/(const BigInt& num1, const BigInt& num2) {
    BigInt q, r;
    num1.div(num2, q, r);
    return q;
}

inline BigInt operator%(const BigInt& num1, const BigInt& num2) {
    BigInt q, r;
    num1.div(num2, q, r);
    return r;
}

inline BigInt operator<<(BigInt num, size_t k) {
    num <<= k;
    return num;
}

inline BigInt operator>>(BigInt num, size_t k) {
    num >>= k;

```

```
        return num;
    }

    inline bool operator==(const BigInt& num1, const BigInt& num2) {
        return num1.cmp(num2) == 0;
    }

    inline bool operator!=(const BigInt& num1, const BigInt& num2) {
        return num1.cmp(num2) != 0;
    }

    inline bool operator<(const BigInt& num1, const BigInt& num2) {
        return num1.cmp(num2) < 0;
    }

    inline bool operator>(const BigInt& num1, const BigInt& num2) {
        return num1.cmp(num2) > 0;
    }

    inline bool operator<=(const BigInt& num1, const BigInt& num2) {
        return num1.cmp(num2) <= 0;
    }

    inline bool operator>=(const BigInt& num1, const BigInt& num2) {
        return num1.cmp(num2) >= 0;
    }
}
```

Додаток Б. Демонстраційна програма main.cpp

```
#include <iostream>
#include "BigInt.hpp"

int main() {
    BigInt z = BigInt::zero();
    BigInt o = BigInt::one();
    BigInt val(42);

    std::cout << "Zero: " << z.to_hex() << "\n";
    std::cout << "One: " << o.to_hex() << "\n";
    std::cout << "Value: " << val.to_hex() << "\n";

    std::string test = "31303130313031303130313031303130313031303130";
    BigInt test_num = BigInt::from_hex(test);
    std::cout << "Test number: " << test_num.to_hex() << "\n";

    BigInt a = BigInt::random();
    BigInt b = BigInt::random();

    std::cout << "\na = " << a.to_hex() << "\n";
    std::cout << "b = " << b.to_hex() << "\n";

    BigInt sum = a + b;
    BigInt diff = a - b;
    BigInt mul = a * b;

    BigInt sum_raw, diff_raw;
    uint32_t carry = a.add(b, sum_raw);
    uint32_t borrow = a.sub(b, diff_raw);
    uint32_t mul_full[2 * WORDS];
    a.mul(b, mul_full);
    uint32_t sq_full[2 * WORDS];
    a.square(sq_full);

    std::cout << " a + b (hex) = " << sum.to_hex() << " (carry: " << carry <<
    "\n";
    std::cout << " a - b (hex) = " << diff.to_hex() << " (borrow: " << borrow
    << ")\n";
    std::cout << " a * b (mod) = " << mul.to_hex() << "\n";
    std::cout << "a * b (full) = " << BigInt::to_hex_adv(mul_full, 2 * WORDS)
    << "\n";
    std::cout << "a^2 (full) = " << BigInt::to_hex_adv(sq_full, 2 * WORDS) <<
    "\n";

    std::cout << "\na cmp a: " << a.cmp(a) << "\n";
    std::cout << "0 cmp 1: " << z.cmp(o) << "\n";
    std::cout << "1 cmp 0: " << o.cmp(z) << "\n";
    std::cout << "a cmp b: " << a.cmp(b) << "\n";

    BigInt max = o << 2047;
    std::cout << "\n1 << 2047 bit_length = " << max.bit_length() << "\n";
```

```

    BigInt of = o << 2048;
    std::cout << "1 << 2048 = " << of.to_hex() << "\n";

    BigInt sh32 = o << 32;
    BigInt res = sh32 >> 32;
    std::cout << "1 << 32 = " << sh32.to_hex() << "\n";
    std::cout << "100000000 >> 32 = " << res.to_hex() << "\n";

    BigInt q = a / b;
    BigInt r = a % b;
    std::cout << "\na / b (hex) = " << q.to_hex() << "\n";
    std::cout << "a mod b (hex) = " << r.to_hex() << "\n";

    BigInt q1 = a / a;
    std::cout << "a / a (hex) = " << q1.to_hex() << "\n";

    BigInt p0, p1, p2;
    BigInt::power(a, z, p0);
    BigInt::power(a, o, p1);
    BigInt::power(BigInt(2), BigInt(10), p2);

    std::cout << "\na^0 (hex) = " << p0.to_hex() << "\n";
    std::cout << "a^1 (hex) = " << p1.to_hex() << "\n";
    std::cout << "2^10 (hex) = " << p2.to_hex() << "\n";

    return 0;
}

```

Додаток В. Модуль верифікації тотожностей test.cpp

```
#include <iostream>
#include "BigInt.hpp"
int main() {
    BigInt one = BigInt::one();
    BigInt a = BigInt::random();
    BigInt b = BigInt::random();
    BigInt c = BigInt::random();

    if (a.cmp(b) < 0) {
        std::swap(a, b);
    }

    std::cout << "a = " << a.to_hex() << "\n\n";
    std::cout << "b = " << b.to_hex() << "\n\n";
    std::cout << "c = " << c.to_hex() << "\n";

    BigInt dist_left = (a + b) * c;
    BigInt dist_right = (a * c) + (b * c);
    std::cout << "(a + b) * c = " << dist_left.to_hex() << "\n";
    std::cout << "(a * c) + (b * c) = " << dist_right.to_hex() << "\n";

    if (dist_left == c * (a + b) && dist_left == dist_right) {
        std::cout << "Distributivity test passed!\n\n";
    } else {
        std::cout << "Distributivity test failed!\n\n";
    }

    BigInt sum = BigInt::zero();
    for (int i = 0; i < 1000; ++i) {
        sum += a;
    }
    BigInt mul = a * BigInt(1000);
    std::cout << "1000 * a = " << mul.to_hex() << "\n";
    std::cout << "sum = " << sum.to_hex() << "\n";
    if (mul == sum) {
        std::cout << "Multiple addition test passed!\n\n";
    } else {
        std::cout << "Multiple addition test failed!\n\n";
    }

    BigInt q = a / b;
    BigInt r = a % b;
    BigInt recon = (b * q) + r;
    std::cout << "q = " << q.to_hex() << "\n";
    std::cout << "r = " << r.to_hex() << "\n";
    std::cout << "b * q + r = " << recon.to_hex() << "\n";
    if (r.cmp(b) < 0 && recon == a) {
        std::cout << "Division invariant test passed!\n\n";
    } else {
        std::cout << "Division invariant test failed!\n\n";
    }
}
```

```

    BigInt a2, b2;
    BigInt::power(a, BigInt(2), a2);
    BigInt::power(b, BigInt(2), b2);
    BigInt diff_left = (a + b) * (a - b);
    BigInt diff_right = a2 - b2;
    std::cout << "(a + b) * (a - b) = " << diff_left.to_hex() << "\n";
    std::cout << "a^2 - b^2 = " << diff_right.to_hex() << "\n";
    if (diff_left == diff_right) {
        std::cout << "Difference of squares test passed!\n\n";
    } else {
        std::cout << "Difference of squares test failed!\n\n";
    }

    BigInt ax, ay, a_xy;
    BigInt::power(a, BigInt(3), ax);
    BigInt::power(a, BigInt(4), ay);
    BigInt::power(a, BigInt(7), a_xy);
    BigInt pow_mul = ax * ay;
    std::cout << "a^3 * a^4 = " << pow_mul.to_hex() << "\n";
    std::cout << "a^7 = " << a_xy.to_hex() << "\n";
    if (a_xy == pow_mul) {
        std::cout << "Power rule test passed!\n\n";
    } else {
        std::cout << "Power rule test failed!\n\n";
    }

    BigInt a_sh = a;
    for (size_t i = WORDS - 4; i < WORDS; ++i) {
        a_sh.digits[i] = 0;
    }
    BigInt shift_res = (a_sh << 100) >> 100;
    std::cout << "a_masked = " << a_sh.to_hex() << "\n";
    std::cout << "(a_sh << 100) >> 100 = " << shift_res.to_hex() << "\n";
    if (shift_res == a_sh) {
        std::cout << "Shift roundtrip test passed!\n\n";
    } else {
        std::cout << "Shift roundtrip test failed!\n\n";
    }

    size_t k = 77;
    BigInt pow2 = one << k;
    BigInt div_pow2 = a / pow2;
    BigInt shift_k = a >> k;
    std::cout << "a / (1 << 77) = " << div_pow2.to_hex() << "\n";
    std::cout << "a >> 77 = " << shift_k.to_hex() << "\n";
    if ((a / pow2) == (a >> k)) {
        std::cout << "Div by 2^k vs shift test passed!\n\n";
    } else {
        std::cout << "Div by 2^k vs shift test failed!\n\n";
    }
}
return 0;
}

```

Додаток Г. Заміри часу bench.cpp

```
#include <iostream>
#include <chrono>
#include <x86intrin.h>
#include "BigInt.hpp"

int main() {
    BigInt a = BigInt::random();
    BigInt b = BigInt::random();

    if (a < b) {
        std::swap(a, b);
    }

    BigInt b_1024 = BigInt::random();
    for (size_t i = 32; i < WORDS; ++i) {
        b_1024.digits[i] = 0;
    }
    b_1024.digits[31] |= 0x80000000;

    BigInt res, q, r, exp(17);
    uint32_t buf[2 * WORDS];

    uint64_t c1 = __rdtsc();
    auto t1 = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < 100000; ++i) a.add(b, res);
    auto t2 = std::chrono::high_resolution_clock::now();
    uint64_t c2 = __rdtsc();
    std::cout << "Add (+): " << (c2 - c1) / 100000 << " cycles | " <<
std::chrono::duration<double, std::micro>(t2 - t1).count() / 100000 << " us\n";

    c1 = __rdtsc();
    t1 = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < 100000; ++i) a.sub(b, res);
    t2 = std::chrono::high_resolution_clock::now();
    c2 = __rdtsc();
    std::cout << "Sub (-): " << (c2 - c1) / 100000 << " cycles | " <<
std::chrono::duration<double, std::micro>(t2 - t1).count() / 100000 << " us\n";

    c1 = __rdtsc();
    t1 = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < 100000; ++i) a.shift_l(37, res);
    t2 = std::chrono::high_resolution_clock::now();
    c2 = __rdtsc();
    std::cout << "Shift (<<): " << (c2 - c1) / 100000 << " cycles | " <<
std::chrono::duration<double, std::micro>(t2 - t1).count() / 100000 << " us\n";

    c1 = __rdtsc();
    t1 = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < 10000; ++i) a.mul(b, buf);
    t2 = std::chrono::high_resolution_clock::now();
    c2 = __rdtsc();
```

```

        std::cout << "Mul (*): " << (c2 - c1) / 10000 << " cycles | " <<
std::chrono::duration<double, std::micro>(t2 - t1).count() / 10000 << " us\n";

        c1 = __rdtsc();
        t1 = std::chrono::high_resolution_clock::now();
        for (int i = 0; i < 10000; ++i) a.square(buf);
        t2 = std::chrono::high_resolution_clock::now();
        c2 = __rdtsc();
        std::cout << "Sqr: " << (c2 - c1) / 10000 << " cycles | " <<
std::chrono::duration<double, std::micro>(t2 - t1).count() / 10000 << " us\n";

        c1 = __rdtsc();
        t1 = std::chrono::high_resolution_clock::now();
        for (int i = 0; i < 100; ++i) a.div(b_1024, q, r);
        t2 = std::chrono::high_resolution_clock::now();
        c2 = __rdtsc();
        std::cout << "Div: " << (c2 - c1) / 100 << " cycles | " <<
std::chrono::duration<double, std::micro>(t2 - t1).count() / 100 << " us\n";

        c1 = __rdtsc();
        t1 = std::chrono::high_resolution_clock::now();
        for (int i = 0; i < 100; ++i) BigInt::power(a, b, res);
        t2 = std::chrono::high_resolution_clock::now();
        c2 = __rdtsc();
        std::cout << "Power: " << (c2 - c1) / 100 << " cycles | " <<
std::chrono::duration<double, std::micro>(t2 - t1).count() / 100 << " us\n";

        return 0;
}

```